

• AULA 6 • CESAR SCHOOL

# Transformer I: self-attention e a arquitetura

**Seu nome**

seuemail@cesar.org.br

Setembro / 2026

14 slides de fluxo • 6 slides de apêndice matemático

# Da mesma palavra em dois contextos até a arquitetura completa.

**01** Contexto: a mesma palavra em duas frases e como a RNN inclui contexto

---

**03** Demonstração: escores, pesos, escala e máscara

---

**05** Exercício: hipótese, verificação e resultado esperado

---

**02** Self-attention como combinação ponderada e a fórmula

---

**04** Máscara causal, multi-head, custo e arquitetura

---

**06** Fechamento: por que representar a posição dos tokens

---

# A mesma palavra em dois contextos

FRASE A

“O **banco** do rio estava cheio.”

margem, banco de areia

FRASE B

“O **banco** do centro estava  
fechado.”

instituição financeira



`e("banco")` – um único vetor estático, idêntico nas duas frases

Como incluir as palavras ao redor na representação?

- O vetor inicial de banco é o mesmo; o contexto precisa entrar no cálculo da nova representação.

# Como a RNN leva o contexto adiante

## RNN — estado oculto

Atualiza um estado a cada token; cada passo depende do anterior.



da posição 1 à 50 = 49 atualizações

## Self-attention — ligação direta

A posição consulta qualquer outra posição na mesma camada.



1 ligação, sem passos intermediários

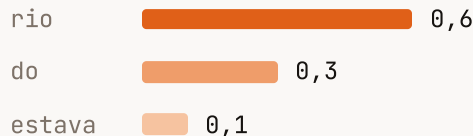
Caminho entre posições:  $O(n)$  na RNN e  $O(1)$  na self-attention de uma camada • [Apêndice A.5](#) • [slide 19](#)

- A RNN inclui contexto passo a passo; a atenção permite uma ligação direta entre posições.

# Como a self-attention combina informações

- Cada token combina informações da própria sequência.
- Os pesos indicam a contribuição de cada conteúdo.
- Os pesos são calculados a partir da entrada.
- Outro contexto pode produzir outra representação.

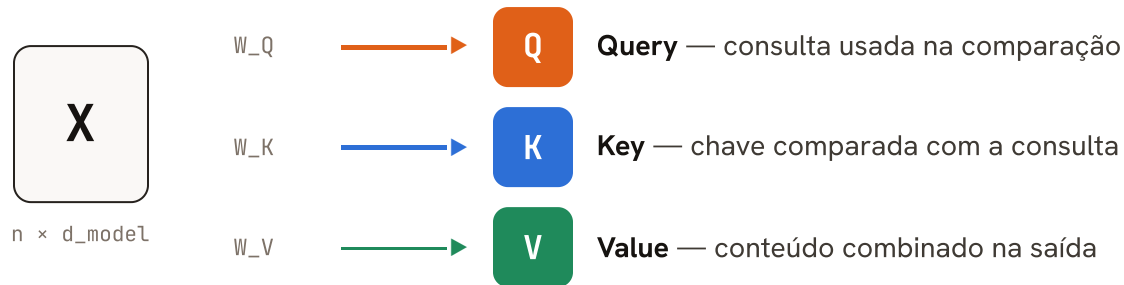
ILUSTRAÇÃO — PESOS HIPOTÉTICOS, NÃO MEDIDOS



Nova representação de **banco** = média ponderada dos conteúdos da sequência.

- A self-attention calcula quanto cada token contribui para a representação dos outros.

# Q, K, V: três papéis, uma entrada



$Q = XW_Q$      $K = XW_K$      $V = XW_V$  — as três projeções vêm da mesma  $X$  • **Apêndice A.1 · slide 15**

- Q e K calculam os pesos; V fornece o conteúdo que será combinado.

# Lendo a fórmula de atenção

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

$QK^T$       escores de comparação entre posições       $/ \sqrt{d_k}$       ajuste de escala;  $d_k$  é a dimensão de Q e K  
 $\text{softmax}$       pesos não negativos que somam 1 por linha       $\cdot V$       média ponderada dos conteúdos

pesos ilustrativos  $0,6 \cdot 0,3 \cdot 0,1$       derivação completa: [Apêndice A.1](#) • [slide 15](#)

- A fórmula compara os tokens, calcula os pesos e combina os conteúdos.

# Por que dividir por $\sqrt{d_k}$

- Escores muito dispersos podem concentrar os pesos.
- **Saturação:** um peso perto de 1 e os demais perto de 0.
- Nessa região, os pesos respondem pouco a pequenos ajustes.
- Dividir por  $\sqrt{d_k}$  compensa o crescimento de escala do exemplo.

| D_K                                                                                                                              | DESVIO-PADRÃO TEÓRICO DE Q·K |
|----------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| 64                                                                                                                               | 8                            |
| 512                                                                                                                              | ≈ 22,6                       |
| Hipóteses: componentes independentes, média zero e variância um.<br>Valores teóricos — separados dos resultados medidos na demo. |                              |

Sob as hipóteses indicadas,  $\text{Var}(q \cdot k) = d_k$  • [Apêndice A.2](#) • [slide 16](#)

- A divisão por raiz de  $d_k$  ajuda a evitar que os pesos fiquem concentrados demais por causa da escala.

# Observando os pesos de atenção

1 Mesmo **banco** na entrada, saídas diferentes.

2 Escores e pesos com e sem a divisão por  $\sqrt{d_k}$ .

3 Máscara causal: pesos futuros iguais a zero.

4 Sem máscara e sem posição: permutar entrada e saída.

```
$ python demo_attention.py --scale --mask --shuffle # tabela impressa durante a execução
```

- Vou comparar as saídas para relacionar cada parte da fórmula com os números do exemplo.

# Máscara causal: usando apenas o contexto disponível

- A posição  $i$  prevê o token  $i+1$ .
- A máscara permite consultar o presente e o passado.
- Os escores futuros recebem  $-\infty$  antes do softmax.
- Depois do softmax, os pesos futuros são zero.
- A soma de cada linha permitida é 1.

ESCORES + M

|   |           |           |           |
|---|-----------|-----------|-----------|
| S | $-\infty$ | $-\infty$ | $-\infty$ |
| S | S         | $-\infty$ | $-\infty$ |
| S | S         | S         | $-\infty$ |
| S | S         | S         | S         |

linhas  $i = 0 \dots 3$  · colunas  $j = 0 \dots 3$

PESOS A

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 0 | 0 |
| . | . | 0 | 0 |
| . | . | . | 0 |
| . | . | . | . |

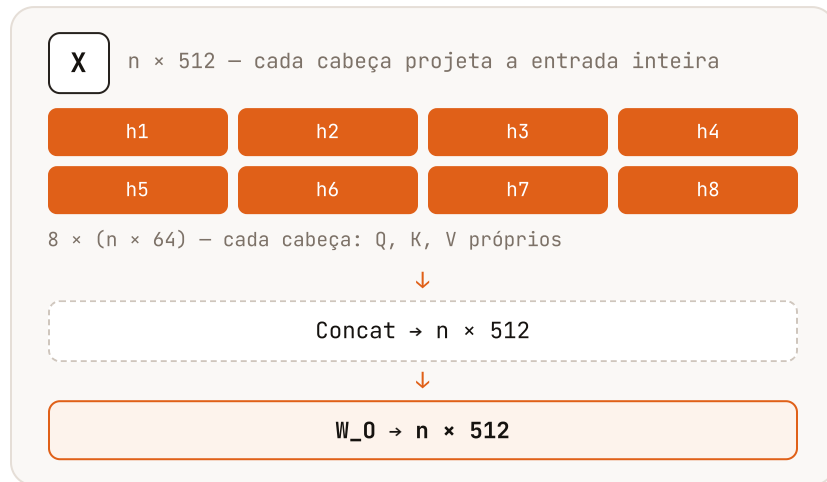
cada linha soma 1

$A = \text{softmax}(QK^T / \text{vd\_k} + M)$ , com  $M$  bloqueando as posições futuras · **Apêndice A.3 · slide 17**

- A máscara causal bloqueia as posições futuras antes de calcular os pesos.

# Multi-head: várias combinações em paralelo

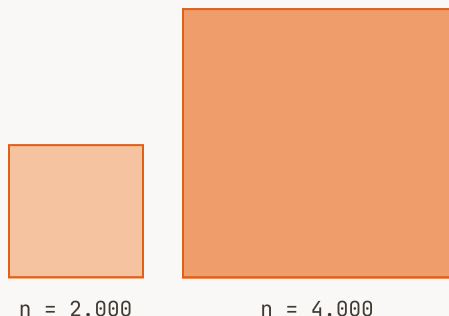
- Cada cabeça tem suas projeções Q, K e V.
- As cabeças calculam combinações em paralelo.
- As saídas são concatenadas e projetadas novamente.
- Exemplo: dimensão 512, oito cabeças de dimensão 64.
- O mapa mostra pesos; a função da cabeça exige investigação.



$d_k = d_v = d_{\text{model}}/h$  •  $\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_0$  • [Apêndice A.4](#) • [slide 18](#)

- Várias cabeças permitem calcular combinações diferentes de informação na mesma camada.

# Como o tamanho do contexto afeta a memória



| TOKENS                                                                                                        | ENTRADAS (N × N)  | 2 BYTES/ENTRADA |
|---------------------------------------------------------------------------------------------------------------|-------------------|-----------------|
| 2.000                                                                                                         | <b>4 milhões</b>  | <b>≈ 8 MB</b>   |
| 4.000                                                                                                         | <b>16 milhões</b> | <b>≈ 32 MB</b>  |
| Por cabeça, por sequência, e só para a matriz de atenção — a estimativa considera armazenar a matriz inteira. |                   |                 |

Cálculo da atenção:  $O(n^2 \cdot d)$  · memória da matriz:  $B \cdot h \cdot n^2 \cdot \text{bytes por elemento}$  · [Apêndice A.5 · slide 19](#)

- Dobrar o contexto quadruplica o número de entradas da matriz de atenção.

# Encoder, decoder e cross-attention

## Encoder

representação da sequência de entrada

Self-attention (sem máscara)

Feed-forward

Residuais + normalização → Aula 7

entrada: "o gato dorme"

## Decoder

produção da sequência de saída

Self-attention causal – consulta à saída já disponível

Cross-attention –  $Q$  do decoder ·  $K, V$  do encoder

Feed-forward

saída: "the cat sleeps"

encoder —————  $K, V$  ► cross-attention do decoder

- Na cross-attention, o decoder faz a consulta e o encoder fornece as chaves e os valores.

# Geração token a token e KV cache

## PASSO 1

Calcular probabilidades para o próximo token.

## PASSO 2

Escolher um token e acrescentá-lo à sequência.

## PASSO 3

Repetir usando o contexto atualizado.

### KV cache — reutilizar chaves e valores anteriores

$K, V \ t_1$   $K, V \ t_2$   $K, V \ t_3$  guardados +  $K, V \ t_4$  calculados agora

Decoder-only: prompt e continuação ficam na mesma sequência.

- O KV cache guarda chaves e valores anteriores para reutilizá-los durante a geração.

# Como investigar três problemas

para cada caso: hipótese · verificação · resultado esperado · 8 min em dupla + 3 min de correção

1

Dimensão 1024: perda de validação  
pior que com 256.

hipótese

verificação

esperado

2

Perda quase zero no treino, mas  
geração incoerente.

hipótese

verificação

esperado

3

Contexto de 2.000 para 4.000:  
memória da GPU excedida.

hipótese

verificação

esperado

- Para cada problema, vamos propor uma hipótese e dizer como testá-la. O enunciado completo está no roteiro.

# Por que ainda precisamos representar a posição

0 cachorro morde o homem

‡ mesma coleção de tokens, outra ordem

0 homem morde o cachorro

Mesmas palavras, sentidos diferentes. **Sem máscara e sem informação posicional**, a saída acompanha a permutação.

Attention(PX) = P · Attention(X) · **Apêndice A.6 · slide 20**

## Aula 7

- Codificação de posição
- Conexões residuais
- Normalização
- Feed-forward

**Leitura:** Vaswani et al., seções 3.1 – 3.3, com apoio de A.2.

- Além do conteúdo de cada token, o modelo precisa de informação sobre sua posição.

# Apêndice matemático

A.1 Derivação de  $\text{softmax}(QK^T/\sqrt{d_k})V$

---

A.2 Por que dividir por  $\sqrt{d_k}$

---

A.3 Máscara causal e a ordem das operações

---

A.4 Multi-head: dimensões e custo

---

A.5 Complexidade  $O(n^2 \cdot d)$

---

A.6 Equivariância a permutação

---

# Derivação de softmax( $QK^T/\sqrt{d_k}$ ) $V$

## NOTAÇÃO

$X \in \mathbb{R}^{(n \times d_{\text{model}})}$   
 $W_Q, W_K \in \mathbb{R}^{(d_{\text{model}} \times d_k)}$   
 $W_V \in \mathbb{R}^{(d_{\text{model}} \times d_v)}$   
 $Q = XW_Q \quad K = XW_K \quad V = XW_V$   
 $q_i, k_j, v_j$  — linhas  $i, j$

## PREMISSAS

As três projeções partem da mesma  $X$  — isto define *self-attention*. Em *cross-attention*,  $Q$  vem de uma sequência e  $K, V$  de outra. Nenhuma premissa distribucional aqui; ela aparece só em A.2.

### 1 Matriz de afinidade

$$S = QK^T \in \mathbb{R}^{(n \times n)} \quad S[i, j] = q_i \cdot k_j$$

### 2 Escala

$$S' = S / \sqrt{d_k} \quad \text{— justificativa em A.2}$$

### 3 Normalização por linha

$$A[i, j] = \exp(S'[i, j]) / \sum_l \exp(S'[i, l])$$

$$\sum_j A[i, j] = 1 \quad \text{para toda linha } i$$

### 4 Agregação

$$AV \in \mathbb{R}^{(n \times d_v)} \quad \text{saída}[i] = \sum_j A[i, j] \cdot v_j$$

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$$

## INTERPRETAÇÃO

A saída é combinação convexa dos values: o centro de massa da nuvem sob esses pesos.

## CASO-LIMITE 1

Escores iguais  $\rightarrow A[i, :] = 1/n$ : média simples de todos os valores.

## CASO-LIMITE 2

Um escore domina  $\rightarrow A[i, j^*] \rightarrow 1$ , saída  $\rightarrow v_{j^*}$ , derivadas do softmax  $\rightarrow 0$ .

# Por que dividir por $\sqrt{d_k}$

Objetivo: mostrar que o desvio-padrão de  $q \cdot k$  cresce com  $\sqrt{d_k}$  sob as hipóteses abaixo — argumento que motiva o fator de escala.

## NOTAÇÃO E PREMISSAS

$q, k \in \mathbb{R}^{(d_k)}$   
 $q_m, k_m$  mutuamente independentes  
 $E[q_m] = 0 \quad \text{Var}(q_m) = 1$

Hipóteses simplificadoras, usadas na geração dos vetores aleatórios da demo. Q e K de uma rede treinada não precisam satisfazê-las: observe também a distribuição real dos escores.

### 1 Média

$$E[q_m k_m] = E[q_m]E[k_m] = 0 \Rightarrow E[q \cdot k] = 0$$

### 2 Variância de cada parcela

$$E[q_m^2 k_m^2] = \text{Var}(q_m) \cdot \text{Var}(k_m) = 1$$

### 3 Variância da soma — aqui a independência é essencial

$$\text{Var}(q \cdot k) = \sum_m \text{Var}(q_m k_m) = d_k \Rightarrow \text{desvio} = \sqrt{d_k}$$

### 4 A correção

$$\text{Var}(q \cdot k / \sqrt{d_k}) = d_k / d_k = 1 \quad \text{em qualquer dimensão}$$

## EFEITO SOBRE O GRADIENTE

$$\partial A_i / \partial z_j = A_i (\delta_{ij} - A_j)$$

Com  $A_i \rightarrow 1$  e  $A_j \rightarrow 0$ , toda a jacobiana tende a zero: o sinal que chega a Q e K por esse caminho pode ficar muito pequeno.

## INTERPRETAÇÃO

$\sqrt{d_k}$  normaliza a escala do produto escalar — não normaliza os vetores  $q$  e  $k$ .

## CASO-LIMITE

Com  $d_k = 1$ ,  $\sqrt{d_k} = 1$  e a divisão não faz nada — não há acúmulo de variância.

## REFERÊNCIA

Vaswani et al., *Attention Is All You Need*, seção 3.2.1 — arXiv 1706.03762.

# Máscara causal e a ordem das operações

## NOTAÇÃO

$M \in \mathbb{R}^{(n \times n)}$ , entradas 0 ou  $-\infty$

$S' = QK^T/\sqrt{d_k}$ , finito

$M[i,j] = 0$  se  $j \leq i$  permitido

$M[i,j] = -\infty$  se  $j > i$  proibido

Índices de 0 a  $n-1$ . A saída da posição  $i$  prevê o token  $i+1$ .

### CASO-LIMITE – PRIMEIRA POSIÇÃO

Para  $i = 0$ , só  $j = 0$  é permitido:  $A[0,0] = 1$  necessariamente.  
Verificação rápida da implementação.

### CORRETO – MASCARAR ANTES DA NORMALIZAÇÃO

$A = \text{softmax}(QK^T/\sqrt{d_k} + M)$

$\exp(-\infty) = 0 \Rightarrow A[i,j] = 0$  exato, para  $j > i$

$\sum_{j \leq i} A[i,j] = 1$  – a massa do futuro é redistribuída

### INCORRETO – ZERAR DEPOIS DA NORMALIZAÇÃO

$\tilde{A}[i,j] = A[i,j] \cdot 1[j \leq i]$

$\sum_j \tilde{A}[i,j] = 1 - \sum_{(j>i)} A[i,j] < 1$

o denominador ainda inclui escores futuros

### COMO VERIFICAR

Antes do dropout: zeros nas posições proibidas e soma  $\approx 1$  por linha. Depois, com dropout desativado, alterar o último token e comparar as saídas anteriores — diferença indica dependência do futuro.

# Multi-head: dimensões e custo

## NOTAÇÃO E DEFINIÇÃO

```
h cabeças · d_k = d_v = d_model/h
W_Q^(l), W_K^(l), W_V^(l) ∈ ℝ^(d_model×d_k)
W_O ∈ ℝ^(d_model×d_model)

head_l = Attention(XW_Q^(l), XW_K^(l), XW_V^(l))
MultiHead(X) = Concat(head_1,...,head_h)W_O
```

A concatenação recompõe  $n \times (h \cdot d_k) = n \times d_{\text{model}}$ , e  $W_O$  remistura os subespaços.

## VERIFICAÇÃO DO CUSTO

```
h · 3 · (d_model · d_k) = 3 · d_model²
h × 0(n² · d_k) = 0(n² · d_model)
```

Mesma ordem de custo de uma única cabeça com  $d_k = d_{\text{model}}$ . Na implementação que guarda os pesos, há  $h$  matrizes  $n \times n$ .

## ORGANIZAÇÃO DOS TENSORES $B$ = tamanho do lote

```
(B, n, d_model)  entrada
(B, n, 3·d_model) projeção qkv combinada
3 × (B, n, d_model) split em q, k, v
(B, n, h, d_k)  view
(B, h, n, d_k)  transpose(1,2) – a cabeça vira lote
(B, h, n, d_k)  atenção, como em A.1
(B, n, h, d_k)  transpose(1,2) de volta
(B, n, d_model) contiguous().view()
(B, n, d_model) projeção W_O
```

## CONTIGUOUS()

Reorganiza os dados para permitir o view. view não troca eixos — a transposição continua necessária.

## CASO-LIMITE

Com  $h = 1$ ,  $d_k = d_{\text{model}}$  e a expressão colapsa em A.1 seguida de  $W_O$ .

# Complexidade $O(n^2 \cdot d)$

## CONTAGEM, TERMO A TERMO

Projeções Q, K, V — três multiplicações  $O(n \cdot d_{\text{model}}^2)$   
 $(n \times d_{\text{model}}) \cdot (d_{\text{model}} \times d_k)$

Afinidade QK<sup>T</sup> —  $n^2$  entradas, cada uma um produto escalar de  $d_k$   $O(n^2 \cdot d_k)$

Softmax — uma exponencial e uma divisão por entrada  $O(n^2)$

Agregação AV —  $(n \times n) \cdot (n \times d_v)$   $O(n^2 \cdot d_v)$

$O(n \cdot d^2)$   
projeções

+

$O(n^2 \cdot d)$   
atenção — domina quando  $n > d$

## MEMÓRIA DA MATRIZ A

memória  $\approx B \cdot h \cdot n^2 \cdot \text{bytes/elemento}$

Dobrar  $n$  quadruplica esse termo, mantendo  $B$ ,  $h$  e a precisão. Guardar  $A$  inteira não é exigência matemática — FlashAttention calcula por blocos.

## COMPARAÇÃO COM A RNN

|         | MEMÓRIA  | PASSOS SEQUENCIAIS       |
|---------|----------|--------------------------|
| RNN     | $O(n)$   | $O(n)$                   |
| Atenção | $O(n^2)$ | <b><math>O(1)</math></b> |

Treinamento, dimensões fixas e implementação que guarda a matriz.  $O(1)$  descreve dependências entre posições, não tempo de execução.

# Equivariância a permutação

Self-attention sem informação posicional e sem máscara é **equivariante** a permutação: permutar as linhas da entrada permuta as linhas da saída da mesma forma, sem alterar nenhum valor.

## NOTAÇÃO

$P \in \{0,1\}^{(n \times n)}$  — matriz de permutação  
exatamente um 1 por linha e por coluna  
 $P^T P = I$     $P$  é ortogonal

## LIMITE DO ENUNCIADO

Uma máscara causal fixa não é permutada junto com  $X$  — a demonstração não se aplica diretamente. Cálculo considerado sem dropout.

**projeções**    $Q' = (PX)W_Q = P(XW_Q) = PQ \cdot K' = PK \quad V' = PV$

**afinidade**    $S' = Q'K'^T = (PQ)(PK)^T = P S P^T$

**escala**    $S'/vd_k = P (S/vd_k) P^T$    divisão por escalar comuta

**softmax**    $\text{softmax}(P S P^T) = P \text{softmax}(S) P^T \Rightarrow A' = P A P^T$

**agregação**    $(P A P^T)(P V) = P A (P^T P) V = P A V = P \cdot \text{saída}$  ■

## LEITURA DO RESULTADO

Nenhum valor muda — só a posição em que aparece. O mecanismo não distingue “o cachorro morde o homem” de “o homem morde o cachorro”.

## CONFIRMAÇÃO EMPÍRICA

Demo 4 (--shuffle) ilustra o teorema; uma permutação não o demonstra para todas.

## CONSEQUÊNCIA

Senoidal, embeddings aprendidos e RoPE — Aula 7. No RoPE,  $Q' = PQ$  deixa de valer.